EX03 - Quadratic Solver - Nested Conditionals and Complex Numbers

Beginner Step-by-Step Guide

This guide follows the real file `ex03_quadratic_solver.py`. It explains the problem first and then the code from the perspective of someone starting Python.

1. What we want to solve

Solve a*x**2+b*x+c=0 including non-quadratic, repeated-root, two-real-root and complex-root cases.

Problem variables / elements

Name	Meaning
a,b,c	equation coefficients
D	discriminant b**2-4*a*c
x	root or roots

Equation or logic on paper

```
a*x**2 + b*x + c = 0
D = b**2 - 4*a*c
x = (-b +/- sqrt(D))/(2*a)
```

Manual check

For a=1,b=-3,c=2: D=1 and roots are 2 and 1. For the active case a=0,b=2,c=1, the equation becomes linear: 2*x+1=0 -> x=-0.5.

2. What we already know and what is new

Already seen:

- variables
- if / elif / else
- np.sqrt()

New in this exercise:

- import numbers
- isinstance()
- nested if statements
- complex()
- conjugate()
- checking by substitution

3. New Python concepts

1. Validate type

Syntax: `isinstance(a, numbers.Real)`

isinstance(object,type) returns True if the object belongs to that type. numbers.Real accepts real values such as int and float.

2. Nested if

Syntax: `if a == 0: if b == 0: ...`

An if inside another is evaluated only after entering the outer block.

3. Complex number

Syntax: `complex(real, imag)`

Python can represent real and imaginary parts. complex(0,2) represents 0+2j.

4. Method

Syntax: `x1.conjugate()`

The dot accesses behavior associated with x1. conjugate() returns the complex conjugate.

4. Code step by step

Step 1. Validate a,b,c

```
if isinstance(a, numbers.Real) == False:
    print(...)
```

The exercise tries to ensure each coefficient is real before solving.

Step 2. Handle a=0 first

```
if a == 0:
    ...
```

If a=0 the x**2 term disappears and the quadratic formula cannot be used because it would divide by 2*a.

Step 3. Compute D in the general case

```
discriminant = b**2 - 4 * a * c
```

D is a derived variable that determines the root type.

Step 4. Choose according to D

```
if discriminant < 0:
    ...
elif discriminant == 0:
    ...
else:
    ...
```

D<0 gives complex roots; D=0 one repeated root; D>0 two real roots.

Step 5. Validate by substitution

```
check_1 = a * x1**2 + b * x1 + c
```

A correct root should make the equation equal to 0 or numerically very close to 0.

5. Execution flow

```
a,b,c
   -> type validation
   -> special cases (a=0, b=0, c=0)
   -> discriminant
   -> roots
   -> substitute back
```

6. Expected result / check

With active case a=0,b=2,c=1, the file returns x=-0.5 and b*x+c=0.0.

7. Common beginner mistakes

- Using the quadratic formula with a=0.
- Confusing = with ==.
- Losing nested-if indentation.
- Not checking roots at the end.

8. If you remember only five things

- == compares; = assigns.
- isinstance() validates types.
- Special cases come before the general case.
- D determines root type.
- Substitution provides an independent validation.

9. Mini-practice

Change one input. Before running, write a comment predicting what should change and why. Then run the file and compare your prediction with the output.

Completion criterion

Move on when you can explain the problem, equation/logic, new Python concepts and execution flow without simply reading the code.